Proyecto Final de Bases de Datos

Solución de datos multisede con arquitectura de microservicios en PostgreSQL

Entrega 1

Descripción del problema, alcance, supuestos y reglas de negocio

Asignatura: Bases de datos

Docente: Daniel Leal

Integrantes: Luis Lopez

Integrantes: Angie Narvaez

Integrantes: Jose Cucanchon

Motor de base de datos: PostgreSQL

Índice

1. Descripción del problema **2**

2. Objetivos **2**

2.1. Objetivo general 2

2.2. Objetivos específicos 2

3. Alcance **3**

3.1. Lo que incluye el proyecto 3

3.2. Lo que no incluye 3

4. Arquitectura de microservicios **3**

4.1. Microservicios y propiedad de los datos 3

4.2. Una instancia con esquemas frente a microservicios reales 4

4.3. Integridad entre microservicios 4

5. Supuestos **5**

6. Reglas de negocio **6**

6.1. Catálogo y terceros 6

6.2. Compras y abastecimiento 7

6.3. Inventarios y transferencias 7

6.4. Ventas y pagos 7

6.5. Identidad, seguridad y distribución 8

6.6. Glosario de estados 8

7. Estrategia de distribución de datos **9**

8. Roles y perfiles de acceso **10**

9. Parámetros del proyecto y volúmenes de datos **10**

10. Trazabilidad con los requerimientos del enunciado **11**

1. Descripción del problema

Una empresa comercial opera varias sedes de venta y un centro de distribución. Cada sede maneja sus productos, inventarios, pedidos, compras y transferencias de mercancía entre almacenes. La organización necesita dos cosas que, en principio, se contraponen:

- **Consultar sus datos localmente**, de modo que cada sede trabaje con su propia información operativa.
- **Obtener reportes consolidados** de toda la empresa, sin perder la separación de responsabilidades entre las distintas áreas (catálogo, compras, inventarios, ventas e identidad).

La solución propuesta consiste en diseñar e implementar en PostgreSQL una capa de datos basada en una **arquitectura de microservicios**. Cada microservicio es responsable lógico de un conjunto definido de entidades y operaciones. La información operativa de mayor volumen (inventarios y ventas) se fragmenta horizontalmente por sede, y la integración se logra mediante vistas y consultas SQL de solo lectura que permiten emitir informes institucionales.

El trabajo se centra en el modelo Entidad–Relación, su transformación al modelo relacional, la normalización hasta tercera forma normal, las restricciones de integridad, los roles y permisos, las vistas, la distribución de los datos y las consultas SQL avanzadas.

2. Objetivos

2.1 Objetivo general

Diseñar e implementar en PostgreSQL la capa de datos de una empresa multisede organizada en microservicios, con fragmentación horizontal por sede, control de acceso basado en roles y una capa de reportes corporativos.

2.2 Objetivos específicos

1. Modelar los datos de la empresa mediante un diagrama Entidad–Relación con cardinalidades y transformarlo a un modelo relacional en tercera forma normal.
2. Asignar cada entidad a un microservicio, definiendo con claridad sus límites de responsabilidad y cómo se relacionan entre sí.
3. Implementar un esquema PostgreSQL por microservicio, con claves primarias y foráneas, restricciones `UNIQUE`, `NOT NULL` y `CHECK`.
4. Fragmentar horizontalmente por sede los inventarios y los pedidos de venta, y reunirlos mediante vistas consolidadas.
5. Definir seis roles con privilegios diferenciados y demostrar accesos permitidos y denegados.
6. Generar datos de prueba coherentes que permitan responder diez vistas y veinte consultas SQL avanzadas.
7. Verificar el funcionamiento de las restricciones, los fragmentos, los permisos y los reportes mediante diez pruebas documentadas.

3. Alcance

3.1 Lo que incluye el proyecto

- Al menos cuatro sedes (una de ellas de distribución) y al menos cuatro almacenes asociados.
- Catálogos de productos, categorías, proveedores, clientes y personal.
- Compras, recepciones, inventarios, transferencias, ventas, pagos y devoluciones.
- Seis esquemas lógicos que simulan seis microservicios.
- Fragmentación horizontal por sede de dos conjuntos operacionales: inventarios y pedidos de venta.
- Consultas individuales por sede y consultas corporativas que integran todos los fragmentos.
- Diez vistas SQL, veinte consultas avanzadas, seis roles y diez pruebas de verificación.
- Documentación técnica y manual de instalación y ejecución con PostgreSQL, pgAdmin o psql.

3.2 Lo que no incluye

Según el enunciado, quedan expresamente fuera del alcance:

- Programar los microservicios, construir una API REST, usar Docker o desarrollar interfaces web o móviles. La arquitectura se diseña y se representa; su capa de datos sí se implementa y se prueba.
- Políticas de seguridad a nivel de fila (RLS), autenticación web u otros mecanismos de seguridad no tratados en clase.
- Mecanismos de sincronización en tiempo real entre fragmentos.
- Pruebas de concurrencia, demostraciones de bloqueo, reversión de transacciones, automatización mediante *triggers* y conciliaciones complejas de movimientos de inventario.

4. Arquitectura de microservicios

4.1 Microservicios y propiedad de los datos

Cada microservicio es propietario lógico de sus datos y se implementa como un esquema de PostgreSQL. La Tabla 1 resume la distribución.

Tabla 1: Microservicios, esquemas y entidades que administran.

Esquema	Microservicio	Entidades	Responsabilidad
`catalogo`	Catálogo y Terceros	`sedes`, `almacenes`, `categorias`, `productos`, `proveedores`, `producto_proveedor`, `clientes`	Datos maestros y terceros

Esquema	Microservicio	Entidades	Responsabilidad
`compras`	Compras y Abastecimiento	`ordenes_compra`, `detalle_orden_compra`, `recepciones_compra`, `detalle_recepcion`	Abastecimiento a proveedores
`inventarios`	Inventarios y Transferencias	`inventarios` (fragmentada), `transferencias`, `detalle_transferencia`	Existencias y traslados entre almacenes
`ventas`	Ventas y Pagos	`pedidos_venta`, `detalle_pedido` y `pagos` (fragmentadas), `devoluciones`, `detalle_devolucion`	Ventas, cobros y devoluciones
`identidad`	Identidad y Accesos	`empleados`, `usuarios`, `auditoria_accesos`	Personal, usuarios y bitácora
`reportes`	Reportes Corporativos	Ninguna tabla; únicamente vistas de solo lectura	Consolidación institucional

4.2 Una instancia con esquemas frente a microservicios reales

En una arquitectura de microservicios real, cada servicio dispone de su propia base de datos, normalmente en una instancia independiente (por ejemplo, una instancia de base de datos en la nube por servicio). Ningún servicio accede directamente a las tablas de otro: la información se intercambia mediante API o eventos, lo que da lugar a consistencia eventual y a que las claves foráneas no puedan cruzar de un servicio a otro.

Este proyecto utiliza **una única instancia de PostgreSQL con un esquema por microservicio**. Se trata de una **simulación académica**: los esquemas aportan separación lógica, pero no independencia, pues todos comparten recursos, punto de falla y despliegue, y nada impediría técnicamente un `JOIN` entre esquemas. Para que la simulación sea fiel a la arquitectura real, se adoptan las siguientes reglas:

- No se definen claves foráneas entre esquemas distintos; esas relaciones se documentan como referencias lógicas (Sección 4.3).
- Un rol de un microservicio no tiene permiso de modificación sobre las tablas de otro.
- El esquema `reportes` contiene únicamente vistas de solo lectura.

Como evolución natural, cada esquema podría migrarse a su propia instancia y las vistas de `reportes` sustituirse por un almacén de datos alimentado mediante eventos.

4.3 Integridad entre microservicios

Las relaciones entre entidades se aseguran de dos formas distintas:

- **Clave foránea (FK) real**: solo entre tablas del mismo esquema (o dentro de una misma tabla particionada). PostgreSQL rechaza valores inexistentes.
- **Referencia lógica**: identificador compartido hacia otro microservicio, sin FK en la base de datos. Su coherencia se valida mediante consultas de control que deben devolver cero filas.

La Tabla 2 lista todas las referencias lógicas del diseño.

Tabla 2: Referencias lógicas entre microservicios (sin FK).

Servicio origen	Tabla y columna	Servicio destino	Entidad referenciada
Identidad	`empleados.id_sede`	Catálogo	`catalogo.sedes`
Compras	`ordenes_compra.id_proveedor`	Catálogo	`catalogo.proveedores`
Compras	`ordenes_compra.id_almacen_destino`	Catálogo	`catalogo.almacenes`
Compras	`ordenes_compra.id_empleado`	Identidad	`identidad.empleados`
Compras	`detalle_orden_compra.id_producto`, `detalle_recepcion.id_producto`	Catálogo	`catalogo.productos`
Inventarios	`inventarios.id_sede` (columna de fragmentación)	Catálogo	`catalogo.sedes`
Inventarios	`inventarios.id_almacen`, `transferencias.id_almacen_origen`, `transferencias.id_almacen_destino`	Catálogo	`catalogo.almacenes`
Inventarios	`inventarios.id_producto`, `detalle_transferencia.id_producto`	Catálogo	`catalogo.productos`
Ventas	`pedidos_venta.id_sede`	Catálogo	`catalogo.sedes`
Ventas	`pedidos_venta.id_almacen`	Catálogo	`catalogo.almacenes`
Ventas	`pedidos_venta.id_cliente`	Catálogo	`catalogo.clientes`
Ventas	`pedidos_venta.id_empleado`	Identidad	`identidad.empleados`
Ventas	`detalle_pedido.id_producto`, `detalle_devolucion.id_producto`	Catálogo	`catalogo.productos`

5. Supuestos

Los siguientes supuestos se adoptan para completar lo que el enunciado deja abierto. Los valores concretos (nombres de sedes, fechas, umbrales y volúmenes) son propuestas del equipo y pueden ajustarse sin alterar la estructura del diseño.

Cód.	Supuesto
S-01	La empresa tiene cuatro sedes con identificadores fijos: 1 Distribución (tipo *Distribucion*), 2 Norte, 3 Sur y 4 Centro (tipo *Venta*). Los ids son fijos porque los fragmentos se definen por este valor.
S-02	Existen al menos cuatro almacenes (se proponen seis). Cada almacén pertenece a una sola sede y es de tipo *Principal*, *Exhibicion* o *Bodega*.
S-03	El centro de distribución también registra pedidos (ventas al por mayor), de modo que los cuatro fragmentos de pedidos contienen datos.

Cód.	Supuesto
S-04	El catálogo (productos, categorías, proveedores y clientes) es único y compartido por todas las sedes.
S-05	Un empleado pertenece a una sola sede y una cuenta de usuario corresponde a un único empleado.
S-06	Se maneja una moneda única; los valores monetarios se almacenan con dos decimales.
S-07	Los datos de prueba cubren seis meses (1 de mayo a 31 de octubre de 2026), más que los cuatro meses mínimos exigidos.
S-08	Para que los resultados sean reproducibles se usa una fecha de corte fija (31 de octubre de 2026) en lugar de la fecha actual del sistema.
S-09	Umbrales de las consultas: un pedido pendiente se considera antiguo si supera 15 días; un producto es de baja rotación si no registra ventas confirmadas en los últimos 90 días.
S-10	Los datos son sintéticos y se generan con un script SQL reproducible (semilla aleatoria fija).
S-11	Los precios y costos unitarios se copian a los detalles de venta y compra al momento de la transacción, para reportar correctamente periodos anteriores.
S-12	El valor `total_registrado` del pedido es una instantánea histórica (dato derivado); se valida contra la suma de su detalle mediante una consulta de control.
S-13	Las reglas que no se imponen con restricciones (por ejemplo, que la cantidad devuelta no supere la vendida) se verifican con consultas de control, pues el enunciado excluye el uso de *triggers*.
S-14	La solución usa una sola instancia de PostgreSQL con esquemas separados; es una simulación académica (Sección 4.2).
S-15	No hay sincronización en tiempo real: las vistas consolidadas leen el estado actual de los fragmentos.

6. Reglas de negocio

6.1 Catálogo y terceros

Cód.	Regla
RN-01	Cada sede se identifica de forma única y tiene un tipo (*Venta* o *Distribucion*); el centro de distribución es una sede más.
RN-02	Todo almacén pertenece exactamente a una sede; una sede tiene uno o más almacenes.
RN-03	Todo producto tiene un SKU único, exactamente una categoría y un estado (*Activo* o *Inactivo*).
RN-04	Un producto puede tener uno o varios proveedores y un proveedor puede surtir varios productos. La relación conserva el costo de referencia y el plazo de entrega en días.

Cód.	Regla
RN-05	Un proveedor se identifica por una identificación única; un cliente, por la combinación única de tipo y número de documento.
RN-06	El `precio_lista` del producto es solo una referencia; no se usa para reportar ventas históricas.

6.2 Compras y abastecimiento

Cód.	Regla
RN-07	Una orden de compra se relaciona con un proveedor, un almacén de destino y un empleado, y contiene una o más líneas (producto, cantidad solicitada y costo unitario).
RN-08	La fecha estimada de entrega no puede ser anterior a la fecha de la orden.
RN-09	Una orden puede tener cero o más recepciones. Cada recepción registra los productos y cantidades recibidos y es parcial o total.
RN-10	Un producto recibido debe pertenecer a la orden, y la cantidad acumulada recibida no debe superar la solicitada (verificado con consulta de control).
RN-11	El costo unitario se conserva en el detalle de la orden como valor histórico.

6.3 Inventarios y transferencias

Cód.	Regla
RN-12	La existencia y el stock mínimo se registran por sede, almacén y producto. Ninguno de los dos puede ser negativo.
RN-13	Cada registro de inventario reside en el fragmento de la sede a la que pertenece su almacén; el almacén referenciado debe ser de esa sede (consulta de control).
RN-14	Una transferencia tiene almacén de origen distinto del de destino y una o más líneas de producto. Las fechas cumplen: solicitud $\leq$ despacho $\leq$ recepción.
RN-15	Coherencia entre estado y fechas: *Solicitado* no tiene despacho ni recepción; *En Transito* tiene despacho y no recepción; *Completado* tiene ambos.
RN-16	La cantidad recibida de una línea es nula mientras la transferencia está en tránsito; las diferencias entre lo enviado y lo recibido se consultan por producto.

6.4 Ventas y pagos

Cód.	Regla
RN-17	Un pedido pertenece a una sede, a un almacén de esa sede, a un cliente y a un empleado de esa sede, y contiene una o más líneas de producto.
RN-18	El precio unitario se copia en el detalle del pedido y no cambia aunque el precio de lista se modifique.

Cód.	Regla
RN-19	`total_registrado` guarda el total del pedido al momento de registrarlo y debe coincidir con la suma de cantidad por precio de sus líneas.
RN-20	Un pedido puede tener varios pagos. El saldo pendiente es el total menos la suma de los pagos en estado *Confirmado*; los pagos pendientes o rechazados no cuentan.
RN-21	Una venta es *confirmada* cuando el pedido está en estado *Pagado* o *Completado*; está *pendiente* cuando está en estado *Pendiente*; los pedidos *Cancelado* no cuentan como venta.
RN-22	Una devolución se asocia a un pedido existente, indica un motivo no vacío y contiene productos que formaron parte de ese pedido, con cantidad devuelta menor o igual a la vendida (consulta de control).

6.5 Identidad, seguridad y distribución

Cód.	Regla
RN-23	Cada usuario pertenece a un único empleado (relación 1:1); un empleado puede no tener usuario.
RN-24	El rol de un usuario es uno de seis: *Administrador*, *Gerencia*, *Inventario*, *Compras*, *Vendedor* o *Auditor*.
RN-25	La auditoría registra el usuario, la fecha y hora, la acción realizada y el objeto consultado.
RN-26	Las contraseñas se almacenan como *hash*; el auditor no tiene acceso a ellas.
RN-27	Un rol de un microservicio no puede modificar las tablas administradas por otro microservicio.
RN-28	Las relaciones entre microservicios se establecen mediante identificadores compartidos; su coherencia se valida con consultas de control.
RN-29	Inventarios, pedidos, detalle de pedido y pagos se fragmentan por sede: cada fila reside en exactamente un fragmento. El identificador de pedido proviene de una única secuencia, por lo que no se repite entre fragmentos.
RN-30	El esquema `reportes` es de solo lectura y consolida información mediante vistas.
RN-31	Los roles operativos no pueden eliminar registros; las anulaciones se registran cambiando el estado.

6.6 Glosario de estados

Entidad	Estados permitidos	Significado relevante
Sedes, almacenes, categorías, productos, proveedores, clientes, empleados y usuarios	Activo, Inactivo	Un producto *activo* es el que puede ofrecerse.
Pedidos de venta	Pendiente, Pagado, Completado, Cancelado	Venta confirmada: Pagado o Completado.

Entidad	Estados permitidos	Significado relevante
Pagos	Confirmado, Pendiente, Rechazado	Solo los confirmados reducen el saldo.
Órdenes de compra	Pendiente, En Camino, Recibido, Cancelado	Compra pendiente: orden con cantidad recibida menor que la solicitada.
Recepciones de compra	Parcialmente, Total	Permite recepciones parciales.
Transferencias	Solicitado, En Transito, Completado, Cancelado	En tránsito: despachada y aún no recibida.
Devoluciones	Solicitado, Aprobado, Rechazado, Procesado	—
Condición del producto devuelto	Nuevo, Defectuoso, Empaque Danado, Usado	—

7. Estrategia de distribución de datos

La fragmentación horizontal se aplica a cuatro tablas del servicio de ventas e inventarios, mediante particionamiento declarativo por lista de valores de `id_sede`. Cada fragmento es una tabla física independiente; no se trata de un simple filtro `WHERE`.

Conjunto fragmentado	Fragmentos	Predicado	Tipo
`inventarios.inventarios`	`inventario_distribucion`, `_norte`, `_sur`, `_centro`	`id_sede` = 1, 2, 3, 4	Horizontal primaria
`ventas.pedidos_venta`	`pedidos_distribucion`, `_norte`, `_sur`, `_centro`	`id_sede` = 1, 2, 3, 4	Horizontal primaria
`ventas.detalle_pedido`	`detalle_pedido_distribucion`, `_norte`, `_sur`, `_centro`	`id_sede` = 1, 2, 3, 4	Derivada del pedido
`ventas.pagos`	`pagos_distribucion`, `_norte`, `_sur`, `_centro`	`id_sede` = 1, 2, 3, 4	Derivada del pedido

Propiedades que debe cumplir la fragmentación:

- **Completitud:** toda fila pertenece a algún fragmento, ya que las cuatro sedes cubren el dominio de `id_sede`.
- **Disyunción:** las listas de valores son mutuamente excluyentes y la clave primaria incluye `id_sede`; además, `id_pedido` sale de una sola secuencia.
- **Reconstrucción:** la unión de los fragmentos (`UNION ALL`) reproduce la relación global; las vistas consolidadas incluyen la columna `fragmento_origen` para identificar la procedencia de cada fila.

Tablas globales del servicio. `transferencias`, `detalle_transferencia`, `devoluciones` y `detalle_devolucion` no se fragmentan. En el caso de las transferencias, cada una involucra

almacenes de dos sedes distintas, por lo que no tiene una sede natural. Estas tablas no tienen aislamiento físico por sede, y esto se documenta como una limitación de la solución.

Control de acceso por sede. Como cada sede tiene sus propias tablas físicas, se restringe el acceso de un usuario a su sede otorgando privilegios solo sobre sus fragmentos y ninguno sobre la tabla padre, sin recurrir a políticas RLS.

8. Roles y perfiles de acceso

Rol	Función y alcance
Administrador	Crea esquemas y usuarios, administra permisos y consulta toda la información.
Gerencia	Consulta reportes consolidados y resultados de las sedes, sin modificar información operativa.
Responsable de inventario	Consulta y actualiza inventarios y transferencias de las sedes autorizadas.
Responsable de compras	Gestiona órdenes de compra y recepciones según las autorizaciones establecidas.
Vendedor	Registra y consulta pedidos y pagos de su sede, sin gestionar tablas de otros servicios.
Auditor	Consulta información y reportes autorizados, sin permisos de modificación y sin acceso a contraseñas.

La matriz detallada de permisos por rol, esquema y operación (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) se presenta en la entrega correspondiente a la seguridad.

9. Parámetros del proyecto y volúmenes de datos

Parámetro	Valor
Periodo de los datos de prueba	1 de mayo a 31 de octubre de 2026
Fecha de corte de las consultas	31 de octubre de 2026 (fija)
Antigüedad de un pedido pendiente (umbral)	Más de 15 días
Baja rotación (sin ventas confirmadas)	90 días
Sedes (incluida la de distribución)	4 (mínimo exigido: 4)
Almacenes	6 propuestos (mínimo exigido: 4)
Categorías / productos	6 / 80 (mínimo exigido: 5 / 80)
Proveedores / clientes / empleados	10 / 100 / 20
Pedidos de venta con detalle y pagos	Alrededor de 320 (mínimo exigido: 300)
Órdenes de compra con recepciones	60
Transferencias / devoluciones	25 / 20

Parámetro	Valor

Los datos de prueba incluyen de forma deliberada sedes con existencias distintas, productos sin ventas recientes, clientes que compran en varias sedes, proveedores con retrasos, pedidos con pagos parciales, compras incompletas y transferencias aún en tránsito, de manera que las veinte consultas produzcan resultados variados.

10. Trazabilidad con los requerimientos del enunciado

N.º	Requerimiento	Cómo se atiende
1	Administración por sede	RN-01, RN-02; `id_sede` en almacenes, empleados y tablas operativas.
2	Microservicios	Sección 4: seis esquemas con responsabilidades definidas.
3	Catálogo	RN-03, RN-04, RN-05; relación N:M `producto_proveedor`.
4	Compras	RN-07, RN-08, RN-11.
5	Recepciones	RN-09, RN-10.
6	Existencias	RN-12, RN-13; inventario fragmentado por sede.
7	Ventas	RN-17, RN-18, RN-19.
8	Pagos	RN-20, RN-21; consulta de pagos confirmados y saldos.
9	Transferencias	RN-14, RN-15, RN-16.
10	Devoluciones	RN-22.
11	Integridad	PK, FK locales, `UNIQUE`, `NOT NULL` y `CHECK` (implementación en la entrega de DDL).
12	Históricos	RN-06, RN-11, RN-18; valores unitarios conservados en los detalles.
13	Permisos	RN-24, RN-27, RN-31; Sección 8.
14	Vistas	Esquema `reportes` (RN-30); diez vistas en la entrega de analítica.
15	Distribución	RN-29; Sección 7.
16	SQL avanzado	Veinte consultas en la entrega de analítica.
17	Datos de prueba	Sección 9.
18	Arquitectura documentada	Sección 4: propiedad de los datos, referencias lógicas e interacciones.